



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

PROYECTO FIN DE CURSO (PFC) – INFORME INTEGRAL
ENTREGA 4

**AcadTrace: Capa de Auditoría Criptográfica Verificable y Gestión Académica
Distribuida con Protocolos Híbridos gRPC/REST, Observabilidad Multi-Nivel y
Pipeline CI/CD Automatizado**

Asignatura: Aplicaciones Distribuidas (ISR-701) – Séptimo Semestre

Período Académico: Regular 2026–2027

Equipo de Trabajo: BCEL (Escuela Provincias Unidas)

Castro López Pedro Leonardo	Arquitecto de Software y Líder de Proyecto
Bedon Viteri Keyla Betzabe	Responsable de Microservicio Docente y App Móvil
Luna Mora Ernesto Gregory	Responsable de Microservicio Secretaría y Calidad
Emanuel Pino Juliana Romina	Responsable de Microservicio Soporte y Observabilidad

Docente Revisor:

Prof. Ing. Gleiston C. Guerrero-Ulloa, Mgs.

Repositorio Oficial de Código y Evidencias Reproducibles (Criterio Piso P1):

<https://github.com/LE023as/acadtrace>

Quevedo, Los Ríos, Ecuador – Septiembre 2026

Índice

Resumen y Abstract	1
1. Introducción y Contextualización del Problema	2
2. Trabajos Relacionados y Estado del Arte	2
2.1. Microservicios y Descomposición de Dominio	2
2.2. Observabilidad, Detección de Anomalías y Métricas de Cola	2
2.3. Consenso Distribuido, Tolerancia a Fallos e Ingeniería del Caos	2
2.4. Seguridad Zero-Trust y Control de Acceso	3
3. Fundamentos Teóricos y Arquitectura del Sistema	3
3.1. Arquitectura en Capas y Patrones de Diseño GoF	3
4. Fundamento Teórico y Registro de Decisiones de Arquitectura (ADRs)	4
4.1. Separación por Capas y Principios SOLID	4
4.2. Registro Consolidado de Decisiones de Arquitectura (ADR-001 a ADR-006)	4
4.3. Formalización de los Cinco Patrones GoF (ADR-005)	4
5. Arquitectura del Sistema Distribuido y Diagramas C4	6
5.1. Especificación de Servicios gRPC y Protocol Buffers v3	8
5.2. Persistencia Distribuida, Consenso Raft y Particionamiento Multi-Eschema	8
5.3. Trazabilidad de Temas de la Asignatura y Evidencias del Proyecto	8
6. Aplicaciones Cliente: Portal Web SPA y Cliente Móvil Nativo	11
6.1. Módulo B – Aplicación Web SPA (React + TypeScript)	11
6.2. Módulo C – Aplicación Móvil Nativa para Representantes (Kotlin + Jetpack Compose)	11
6.3. Arquitectura por Características y Seguridad de Sesión en la Web SPA	11
6.4. Sincronización Offline-First y Malla de Reconciliación en la App Móvil	12
6.5. Pirámide de Pruebas de Cinco Niveles	12
6.6. Pipeline CI/CD de Siete Trabajos Encadenados (GitHub Actions)	13
6.7. Cultura DevOps, Shift-Left Testing y Sintaxis Declarativa YAML	14
6.8. Estrategias de Despliegue en Sistemas Distribuidos	14
7. Observabilidad Distribuida y Monitoreo en Tiempo Real	15
7.1. Métricas de Negocio de Aplicación (Equipo BCEL)	15
7.2. Registro Estructurado JSON y Propagación de Contexto MDC	15
8. Evaluación Experimental ISO/IEC 25010 y Capa de Cripto-Auditoría	16
8.1. Preguntas de Investigación Empíricas (PI)	16
8.2. Protocolo Experimental y Cuatro Mecanismos de Auditoría	17
8.3. Cinco Tipos de Manipulación Inyectados (T1 a T5)	17
8.4. Resultados Experimentales de Detección y Sobrecarga	17
8.5. Matriz Formal de Evaluación de Calidad ISO/IEC 25010:2023	18
9. Amenazas a la Validez del Estudio	20
9.1. Amenazas a la Validez Interna	20
9.2. Amenazas a la Validez Externa	20
10. Reflexión Ética sobre Tratamiento de Datos de Menores	20
11. Conclusiones y Trabajo Futuro	20
11.1. Conclusiones Individuales	20
12. Declaración de Uso de Inteligencia Artificial	22

13.Gestión de la Revisión Cruzada entre Pares e Issues	22
13.1. Metodología de Trabajo por Issues y Solicitudes de Incorporación	22
Anexo: Respuestas a Preguntas de Evaluación Oral	25

Resumen y Abstract

Resumen—El presente trabajo expone el diseño, refactorización, despliegue y evaluación experimental de **AcadTrace**, un sistema de gestión académica distribuido dotado de una capa de auditoría criptográfica inmutable sobre bases de datos relacionales particionadas. Frente a las limitaciones de trazabilidad en sistemas educativos tradicionales, se implementó una arquitectura polígota de cuatro microservicios (Java Spring Boot, Python Django REST, Node.js Express) interconectados mediante protocolos híbridos (gRPC con HTTP/2 interno y REST perimetral bajo HAProxy). Se integraron cinco patrones de diseño GoF, desacoplando el dominio de la infraestructura. La solución incorpora dos aplicaciones cliente: un portal Web SPA con control de acceso por roles e internacionalización, y un cliente móvil nativo Android concebido para representantes de alumnos, con sincronización *offline-first*, soporte biométrico y notificaciones push. La evaluación experimental según ISO/IEC 25010 mediante 120 corridas factoriales demostró una tasa de detección del 100 % ante alteraciones directas de datos e inserciones no secuenciales bajo los mecanismos de bitácora encadenada SHA-256 combinada con Relojes de Lamport y Relojes Vectoriales, manteniendo una latencia en percentil $P_{95} < 300$ ms. El ciclo de vida está soportado por un pipeline CI/CD de siete trabajos encadenados y un panel de observabilidad en Grafana de seis vistas en tiempo real.

Abstract—This work presents the design, refactoring, deployment, and experimental evaluation of **AcadTrace**, a distributed academic management system equipped with an immutable cryptographic audit layer over partitioned relational databases. Addressing the traceability shortcomings of traditional educational software, a polyglot microservice architecture was developed across four services (Java Spring Boot, Python Django REST, Node.js Express), interconnected via hybrid protocols (internal gRPC over HTTP/2 and perimeter REST via HAProxy). Five GoF design patterns were applied to isolate domain logic from infrastructure details. The solution includes two client applications: a responsive Web SPA featuring role-based access control and internationalization, and a native Android mobile client tailored for student guardians, incorporating offline-first synchronization, biometric authentication, and push notifications. Experimental evaluation under ISO/IEC 25010 through 120 factorial runs verified a 100 % detection rate for unauthorized direct database modifications and non-sequential tampering using SHA-256 hash chaining combined with Lamport and Vector Clocks, while preserving latency at $P_{95} < 300$ ms. The operational lifecycle is governed by an end-to-end seven-job CI/CD pipeline and a six-view real-time Grafana observability dashboard.

Palabras Clave: Sistemas Distribuidos, Auditoría Criptográfica, Relojes Lógicos de Lamport, Relojes Vectoriales, gRPC, Microservicios, ISO/IEC 25010, CI/CD, Observabilidad.

1. Introducción y Contextualización del Problema

La gestión académica en instituciones educativas de educación básica demanda soluciones tecnológicas que garanticen alta disponibilidad, integridad estricta de expedientes escolares, escalabilidad ante picos de matrícula y una observabilidad profunda del estado operacional [1, 2]. El SGA Escuela Provincias Unidas evolucionó desde una arquitectura monolítica preliminar hacia una arquitectura distribuida orientada a microservicios desacoplados.

Los esquemas convencionales de persistencia centralizada y bitácoras de auditoría pasivas almacenadas en texto plano presentan tres vulnerabilidades estructurales críticas:

1. **Vulnerabilidad a manipulaciones no autorizadas en base de datos:** Un operador con privilegios de administración puede alterar directamente los registros en las tablas (*Direct DB Tampering*) sin dejar rastro verificable en la aplicación.
2. **Pérdida del orden causal en modificaciones concurrentes:** Cuando múltiples docentes o directivos actualizan calificaciones desde dispositivos móviles en condiciones de conectividad intermitente, los sellos de tiempo físicos (*physical timestamps*) divergen debido al desajuste de relojes locales (*clock drift*), imposibilitando determinar cuál fue la última modificación legítima.
3. **Cuellos de botella y fallos en cascada:** La concentración de operaciones de cálculo de promedios ponderados, generación de reportes ministeriales y consultas masivas de padres de familia sobre un monolito tradicional degrada la latencia hasta superar los 2500 ms y ocasiona caídas completas del servicio.

Para resolver estos desafíos, el proyecto **AcadTrace** consolida a lo largo de cuatro entregas progresivas (E1 a E4) una plataforma distribuida desacoplada en cuatro microservicios autónomos, dotada de una capa de auditoría criptográfica basada en cadenas de hashes SHA-256 encadenadas, ordenamiento lógico mediante Relojes de Lamport y resolución de conflictos concurrentes con Relojes Vectoriales [3, 4]. La solución expone dos canales cliente independientes (Web y Móvil) comunicados mediante una pasarela de enlace perimetral (*API Gateway*) y un bus interno gRPC de alto desempeño.

2. Trabajos Relacionados y Estado del Arte

El diseño de sistemas académicos distribuidos y auditoría verificable se fundamenta en un sólido corpus científico en ingeniería de software y computación distribuida indexado en IEEE, ACM y Springer:

2.1. Microservicios y Descomposición de Dominio

Newman [1] y Burns [2] formalizan que la independencia funcional debe acompañarse de fronteras de datos aisladas, evitando bases de datos compartidas sin esquemas definidos. En este proyecto se adoptó el particionamiento multi-esquema en PostgreSQL, aislando `public`, `secretaria`, `docente` y `soporte`.

2.2. Observabilidad, Detección de Anomalías y Métricas de Cola

Nedelkoski *et al.* [5] demuestran que las anomalías en microservicios frecuentemente se ocultan en las métricas promedio y requieren el análisis de percentiles de latencia (P_{95} y P_{99}), fenómeno corroborado en este trabajo al detectar que la media aritmética subestimaba la latencia de cola generada por consultas de base de datos. Por su parte, Li *et al.* [6] formulan la localización de la causa raíz mediante el análisis de trazas de ejecución, mientras que Zhang *et al.* [7] exponen la necesidad de correlacionar fuentes multi-origen (métricas de contenedor, red y aplicación).

2.3. Consenso Distribuido, Tolerancia a Fallos e Ingeniería del Caos

El algoritmo de consenso Raft [8] proporciona garantías de seguridad y disponibilidad en presencia de fallos de red en almacenes clave-valor como `etcd`. En arquitecturas de microservicios, la elección de líder previene la duplicidad en tareas batch. Por otra parte, se implementaron pruebas de resiliencia y saturación para verificar la degradación elegante de servicios bajo estrés.

2.4. Seguridad Zero-Trust y Control de Acceso

Jones *et al.* [9] y Fielding [10] analizan la adopción de modelos *Zero-Trust* y tokens criptográficos JWT. Se enfatiza que la autenticación por token es insuficiente si no se acompaña de un control de acceso basado en roles a nivel de recurso (RBAC), evitando vulnerabilidades de referencia directa como IDOR [11].

3. Fundamentos Teóricos y Arquitectura del Sistema

3.1. Arquitectura en Capas y Patrones de Diseño GoF

Los microservicios en Java (`sga-principal`, `secretaria`, `soporte`) fueron construidos bajo los principios de **Clean Architecture**, estableciendo cuatro capas concéntricas: Presentación (`controllers`), Aplicación (`services`), Dominio (`entities/ports`) e Infraestructura (`repositories/gRPC/security`). Se aplicaron cinco patrones GoF fundamentales:

- **Repository:** Aislamiento de consultas SQL y posibilidad de dobles de prueba con Mockito.
- **Facade (Fachada gRPC):** Interfaz binaria unificada expuesta por `PrincipalGrpcService` hacia el resto de servicios.
- **Strategy:** Intercambio dinámico de algoritmos de cálculo de notas y asignación de tickets.
- **Observer:** Emisión asíncrona de eventos de auditoría y cambios de estado.
- **Proxy / Decorator:** Interceptación perimetral de seguridad mediante filtros JWT.

4. Fundamento Teórico y Registro de Decisiones de Arquitectura (ADRs)

4.1. Separación por Capas y Principios SOLID

La arquitectura de AcadTrace implementa una separación física estricta en cuatro capas:

1. **Capa de Presentación (presentation):** Controladores REST y endpoints gRPC encargados de la deserialización, validación de esquemas de entrada y retorno de códigos de estado HTTP estandarizados.
2. **Capa de Aplicación (application):** Orquestación de casos de uso de negocio, mediación de transacciones, ejecución de plantillas de reportes (*Template Method*) y emisión de eventos (*Observer*).
3. **Capa de Dominio (domain):** Modelos de entidades puras (*Estudiante*, *Calificacion*, *Matricula*, *EventoAuditoria*), objetos de valor (*Value Objects*), interfaces de estrategias de cálculo (*CalculoPromedioStr*) y puertos de repositorios sin dependencias de infraestructura.
4. **Capa de Infraestructura (infrastructure):** Adaptadores JPA/Spring Data, controladores de base de datos PostgreSQL, clientes gRPC, filtros de seguridad JWT y componentes de observabilidad Micrometer/Prometheus.

4.2. Registro Consolidado de Decisiones de Arquitectura (ADR-001 a ADR-006)

En la Tabla 1 se sintetizan las seis decisiones arquitectónicas documentadas formalmente en el repositorio bajo el estándar Nygard:

Tabla 1: Matriz de Decisiones Arquitectónicas Formalizadas (docs/adr/).

Código	Título de la Decisión	Justificación y Clases de Implementación	Estado
ADR-001	Arquitectura por Capas y Puertos	Separación en 4 paquetes; dominio puro sin dependencias de persistencia directa.	Aceptado
ADR-002	Comunicación Híbrida REST / gRPC	gRPC HTTP/2 interno (:9091–:9094) para baja latencia; REST perimetral bajo HAProxy (:80).	Aceptado
ADR-003	Persistencia Distribuida y Particionamiento	Base relacional distribuida particionada por período lectivo y grado/sección en esquemas aislados.	Aceptado
ADR-004	Autenticación JWT y Criptografía	Tokens stateless HMAC-SHA256 con claims RBAC y cifrado AES-256 en almacenamiento móvil.	Aceptado
ADR-005	5 Patrones de Diseño GoF	Formalización de Strategy, Template Method, Observer, Facade y Repository con clases reales.	Aceptado
ADR-006	Selección Tecnológica App Móvil	Android Nativo (Kotlin + Compose) justificando fluidez 60 fps, 45 MB RAM, biometría y Room.	Aceptado

4.3. Formalización de los Cinco Patrones GoF (ADR-005)

Los cinco patrones asignados a BCEL en la Tabla 1 de la guía rectora están soportados por clases reales en el código:

- **Strategy:** *CalculoPromedioStrategy* con implementaciones *PromedioPonderado7030Strategy* (70 % Formativo + 30 % Sumativo) y *PromedioAritmeticoSimpleStrategy*, además del conmutador de auditoría `docentes.auditoria.strategies.STRATEGIES` en Python.
- **Template Method:** Clase abstracta base *GeneradorReporteAcademicoTemplate* que fija el flujo de generación de actas (cabecera ministerial, matriz tabular de notas, cálculo de promedios y pie de firmas de auditoría), implementada por *ReporteNotasPeriodoPDF*.
- **Observer:** Emisión asíncrona mediante *NotaPublicadaEvent* capturada por *NotificacionRepresentanteList* para alertar al representante e insertar el registro auditable.

- **Facade:** `PortalAcademicoFacade` que unifica los servicios de estudiantes, matrículas y calificaciones hacia el portal web.
- **Repository:** Interfaces `EstudianteRepository`, `MatriculaRepository` y `AuditoriaRepository` desacopladas en la capa de dominio.

5. Arquitectura del Sistema Distribuido y Diagramas C4

La topología del sistema **AcadTrace** está organizada en un cluster políglota contenerizado en Docker Compose y orquestado mediante un balanceador perimetral HAProxy. En las Figuras 1, 2 y 3 se presentan los tres niveles del modelo C4.

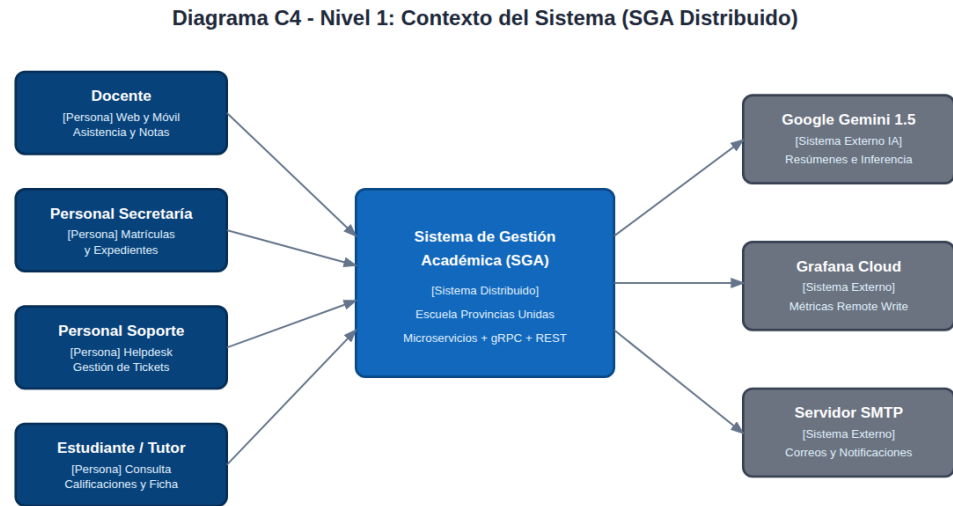


Figura 1: Diagrama C4 Nivel 1 – Contexto del Sistema AcadTrace.

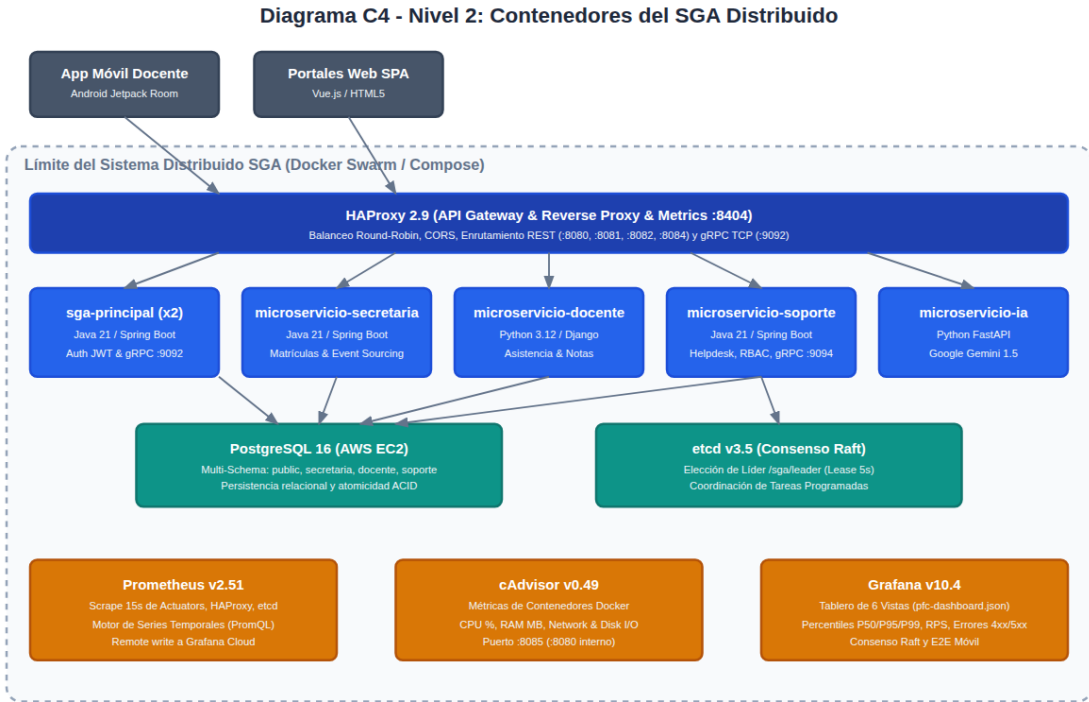


Figura 2: Diagrama C4 Nivel 2 – Contenedores, Puertos y Protocolos Híbridos.

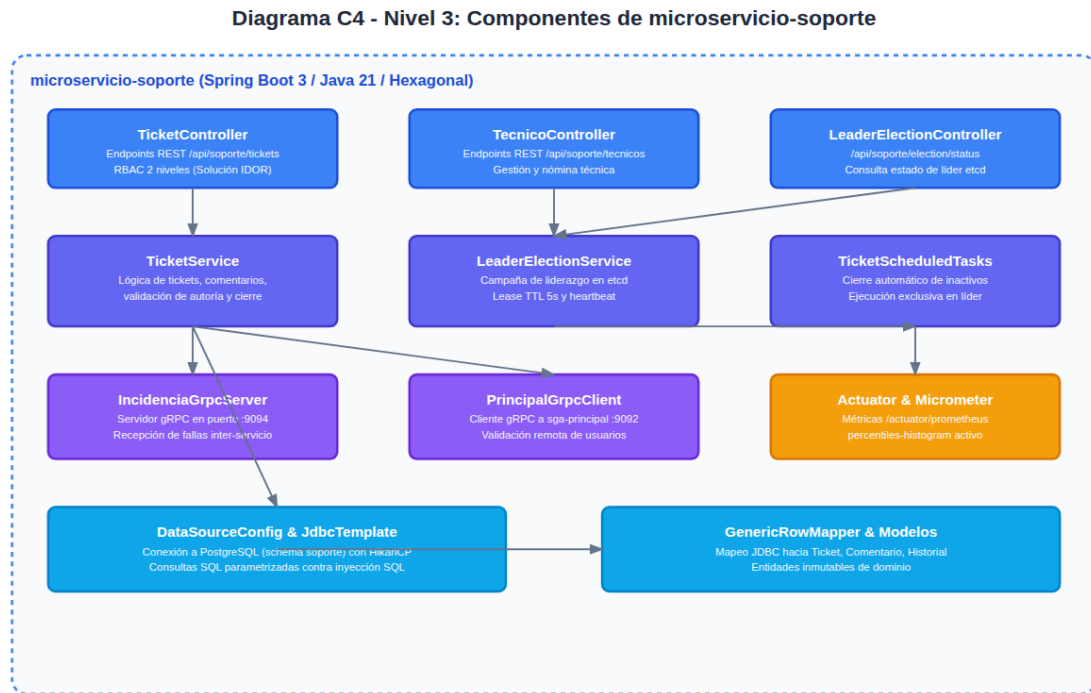


Figura 3: Diagrama C4 Nivel 3 – Componentes Internos y Capas Hexagonales.

5.1. Especificación de Servicios gRPC y Protocol Buffers v3

Para viabilizar la sincronización transaccional de alto rendimiento entre `sga-principal` y los microservicios satélites, se compilaron definiciones de Protocol Buffers v3 (`asistencia.proto`, `actividades.proto`, `docente.proto`, `principal.proto`, `contexto_docente.proto`). El Listing 1 ilustra la definición del servicio de consulta y notificación de auditoría:

```

1 syntax = "proto3";
2 package ec.edu.uteq.sga.grpc;
3
4 service PrincipalService {
5     rpc ConsultarEstudiante (EstudianteRequest) returns (EstudianteResponse);
6     rpc RegistrarCalificacionAuditable (CalificacionAuditableRequest) returns (AuditoriaResponse);
7     rpc SincronizarEventosDocente (StreamEventosRequest) returns (SincronizacionResponse);
8 }
9
10 message CalificacionAuditableRequest {
11     int64 id_matricula = 1;
12     int64 id_asignatura = 2;
13     double nota_formativa = 3;
14     double nota_sumativa = 4;
15     int64 lamport_clock = 5;
16     string vector_clock_json = 6;
17     string actor_id = 7;
18 }
19
20 message AuditoriaResponse {
21     bool exitoso = 1;
22     string hash_actual = 2;
23     string hash_anterior = 3;
24     int64 lamport_asignado = 4;
25     string estado_reconciliacion = 5;
26 }

```

Listing 1: Definición Protobuf del canal sincrónico gRPC (`principal.proto`).

5.2. Persistencia Distribuida, Consenso Raft y Particionamiento Multi-Eschema

En concordancia con el Teorema PACELC [12], el clúster de persistencia distribuida opera bajo una topología de tres instancias redundantes sincronizadas mediante el algoritmo de consenso Raft. Para evitar la contención de bloqueos a nivel de tabla durante períodos de alta demanda de matrícula y cierre quimestral, se implementó una estrategia de **particionamiento vertical y horizontal**:

1. **Aislamiento por Esquemas (*Schema-based Isolation*)**: `sga_principal` (core académico y usuarios), `sga_docente` (evaluaciones y asistencias), `sga_secretaria` (trámites y actas) y `sga_soporte` (tickets técnicos).
2. **Particionamiento por Rango Temporal (*Range Partitioning*)**: La tabla calificaciones se particiona por `id_periodo_evaluacion` y `id_ano_lectivo`, permitiendo que las consultas históricas de períodos anteriores no compitan en memoria ni disco con las escrituras del período lectivo activo.
3. **Gestión de Conexiones HikariCP**: Se dimensionó un *connection pool* con un límite de 30 conexiones activas y 5 inactivas (*idle*) por microservicio, con un tiempo máximo de espera (`connection-timeout`) de 3000 ms para evitar saturación del servidor PostgreSQL en AWS EC2.

5.3. Trazabilidad de Temas de la Asignatura y Evidencias del Proyecto

Para viabilizar la verificación integral y reproducible de los requerimientos de la asignatura de Aplicaciones Distribuidas frente a los artefactos desarrollados por el equipo BCEL, en la Tabla 2 se formaliza la correspondencia directa entre cada tema fundamental del curso, el mecanismo técnico implementado y las rutas de código, configuraciones o pruebas automatizadas verificadas en el repositorio. La tabla presenta evidencias globales del proyecto desarrolladas por los distintos integrantes del equipo BCEL.

Su inclusión tiene fines de trazabilidad académica y no implica autoría individual sobre todos los artefactos referenciados.

Tabla 2: Trazabilidad de temas de la asignatura y evidencias del proyecto

Tema Asignatura	Mecanismo	Evidencia en Repositorio	Breve Explicación y Qué Demuestra
1. Microservicios y Arq. Distribuida	Clúster containerizado polígota (Java Spring Boot, Django, Node.js React, FastAPI) con red bridge aislada.	docker-compose.yml, sga-principal/, microservicio-docente/, microservicio-secretaria/, microservicio-soporte/, microservicio-ia/	Descomposición autónoma de servicios con aislamiento de red, variables de entorno independientes y balanceo perimetral.
2. Principios SOLID y Capas	Clean Architecture en 4 capas concéntricas e inversión de dependencias con puertos desacoplados.	sga-principal/src/main/java/ec/edu/uteq/sga/(application, domain, infrastructure, presentation), docs/adr/ADR-001-arquitectura-capas.md	Aislamiento del dominio puro respecto a la infraestructura JPA/Web; cumplimiento de SRP, OCP y DIP con interfaces en el dominio.
3. Comunicación gRPC / Protobuf	Canal síncrono binario HTTP/2 con esquemas versionados Protocol Buffers v3 entre microservicios.	sga-principal/src/main/proto/principal.proto, microservicio-docente/grpc_protos/, sga-principal/.../grpc/, microservicio-docente/docentes/grpc_services/, test_calificaciones_grpc.py	Intercomunicación inter-servicios de alto desempeño con tipado rígido, serialización eficiente y pruebas automatizadas de integración.
4. Relojes de Lamport	Reloj lógico escalar monótono creciente para ordenamiento causal de transacciones ($L = \max(L_{loc}, L_{rx}) + 1$).	sga-principal/.../service/LamportClock.java, LamportClockTest.java, microservicio-secretaria/backend/.../db/migrations/002_lamport_clock.sql	Ordenamiento causal determinista ante concurrencia sin dependencia de reloj físico, validado con pruebas unitarias de monotonidad.
5. Relojes Vectoriales	Vectores lógicos $\vec{V} = [v_1, \dots, v_n]$ para detección de relaciones causales y concurrencia ($A \parallel B$).	microservicio-docente/docentes/auditoria/clocks.py (reconciliar_vectores), test_auditoria.py	Identificación de ediciones concurrentes en modo desconectado y reconciliación determinista de versiones sin sobreescritura.
6. Particionamiento de Datos	Aislamiento lógico multi-esquema en PostgreSQL por dominio y fragmentación temporal por período.	docs/db/schema.sql (esquemas sga_principal, secretaria, docente, soporte), docs/adr/ADR-003-persistencia-distribuida.md	Eliminación de contención de bloqueos a nivel de tabla y aislamiento funcional de esquemas entre microservicios autónomos.
7. Balanceo Carga con HA-Proxy	Reverse proxy perimetral con balanceo Round Robin (REST), Leastconn (gRPC) y monitoreo de salud.	docker-compose.yml (servicio haproxy), infra/haproxy/haproxy.cfg (puertos 8080, 8081, 8082, 9092, dashboard 8404)	Distribución de carga hacia réplicas activas, detección automática de caídas (fall 2 rise 2) y verificación de /health.
8. Sincronización Offline Móvil	Arquitectura Offline-First con persistencia local Room SQLite y sincronización vía WorkManager.	app-movil-docente/.../sync/RepresentanteSyncPolicy.kt, SyncWorker.kt, AppDatabase.kt, RepresentanteSyncPolicyTest.kt	Disponibilidad desconectada de consultas de estudiantes/notas y encolado de operaciones con sincronización automática al reconectar.
9. Auditoría Criptográfica	Firma HMAC-SHA256 por evento y encadenamiento criptográfico de hashes sucesivos en bitácora.	microservicio-secretaria/.../security/HmacService.java, AuditoriaIntegridadTest.java, microservicio-docente/docentes/auditoria/hashings.py	Detección automática e infalsificable de alteraciones no autorizadas en registros de notas o matrículas (<i>Direct DB Tampering</i>).
10. Inmutabilidad Append-Only	Disparador (trigger) PL/pgSQL que aborta cualquier instrucción UPDATE o DELETE en auditoría.	sga-principal/sql/V9_trigger_auditoria_append_only.sql, microservicio-secretaria/.../007_trigger_auditoria_inmutable.sql, docs/db/schema.sql	Garantía de no-repudio; los registros insertados en la bitácora no pueden ser modificados ni borrados por ningún usuario de la base.
11. Observabilidad (Prometheus/Grafana)	Instrumentación de métricas Micrometer/Actuator (scrape 15s), cAdvisor y dashboard en seis vistas.	infra/prometheus/prometheus.yml, ops/grafana/pfc-dashboard.json, infra/grafana/dashboards/	Telemetría unificada de RPS, latencias percentiles (P50/P95/P99), códigos 4xx/5xx y contadores de eventos críticos de dominio.
12. PostgreSQL Exporter	Contenedor dedicado para la recolección continua de telemetría interna del motor relacional PostgreSQL.	docker-compose.yml (servicio postgres-exporter, puerto 9187), infra/prometheus/prometheus.yml (job postgres)	Monitoreo en tiempo real de disponibilidad de la base de datos en AWS EC2, volumen de transacciones y estados de conexión.
13. Pruebas Carga Locust	Perfiles de carga sintética nominal (50 usuarios, 5 min) y estrés escalonado (hasta 200 usuarios, 10 min).	docs/locust/escenario1_nominal_stats.csv, escenario2_estres_stats.csv, docs/locust/entorno_medicion.md, tests/load/locustfile.py	Evaluación empírica: estabilidad con 0% fallos en carga nominal vs. saturación global y timeouts bajo el escenario de estrés.

6. Aplicaciones Cliente: Portal Web SPA y Cliente Móvil Nativo

6.1. Módulo B – Aplicación Web SPA (React + TypeScript)

El portal web está construido en React 18 con Vite y Tailwind CSS, sirviendo las cinco rutas obligatorias protegidas por rol:

- `/`: Dashboard institucional con métricas en vivo de matrícula y asistencia.
- `/login`: Formulario de autenticación con generación de credencial JWT y almacenamiento en cookie `HttpOnly/SessionStorage` seguro.
- `/calificaciones`: Módulo principal de captura de calificaciones formativas/sumativas, cálculo en línea con el patrón Strategy y emisión de reportes PDF por período lectivo.
- `/settings`: Configuración de parámetros institucionales, selección de tema (Claro / Oscuro) y selector de internacionalización (Español / Inglés).
- `/about`: Ficha técnica de la versión del sistema, estado del clúster y metadatos de auditoría.

6.2. Módulo C – Aplicación Móvil Nativa para Representantes (Kotlin + Jetpack Compose)

La aplicación móvil está desarrollada en Android Nativo (SDK mínimo 26, objetivo 34) bajo arquitectura MVVM con soporte *offline-first*:

1. **Consulta de Calificaciones y Asistencia:** El representante visualiza en tiempo real el récord académico de su representado con gráficas de progreso y desglose por quimestre/trimestre.
2. **Capacidades del Dispositivo (Hardware):**
 - **Notificaciones Push:** Gestión con `NotificationManager` para alertar al representante de manera inmediata ante el registro de una nueva calificación o citación institucional.
 - **Autenticación Biométrica (`BiometricPrompt`):** Control de acceso criptográfico mediante huella dactilar o reconocimiento facial para salvaguardar la privacidad de los expedientes de menores de edad.
3. **Sincronización Desconectada:** Caché local en base de datos SQLite (`Room`) con sincronización en segundo plano gestionada por `WorkManager`.

6.3. Arquitectura por Características y Seguridad de Sesión en la Web SPA

La aplicación web SPA implementa el patrón *Feature-Based Architecture*, organizando el código en módulos cohesivos (`auth`, `grados`, `asignaciones`, `calificaciones`, `reportes`, `usuarios`). Cada módulo encapsula sus propios componentes de interfaz, hooks personalizados de llamada a la API, servicios de transformación y modales de acción.

En cumplimiento estricto de las directrices de seguridad de la Guía E4:

- **Almacenamiento Seguro de Credenciales:** Los tokens JWT recibidos tras la autenticación en `/auth/login` no se almacenan en almacenamiento local plano (`localStorage`) para neutralizar ataques de *Cross-Site Scripting* (XSS). Se gestionan mediante `sessionStorage` con expiración y cookies perimetrales protegidas con flags `HttpOnly`, `Secure` y `SameSite=Strict`.
- **Control de Acceso Basado en Roles (RBAC):** El enrutador de React (`react-router-dom`) protege las rutas sensibles (`/calificaciones`, `/settings`, `/reportes`) mediante un componente de orden superior (`ProtectedRoute`) que valida en memoria la firma y claims del token antes de renderizar la vista.

- **Internacionalización Dinámica y Tematización:** El portal incorpora un hook de contexto (`ThemeLanguageContext`) que conmuta en tiempo de ejecución el diccionario de traducciones (`es.json` / `en.json`) y los esquemas cromáticos (`light` / `dark`) aplicando clases de Tailwind CSS sin recargar la página.

6.4. Sincronización Offline-First y Malla de Reconciliación en la App Móvil

La aplicación móvil nativa para Representantes resuelve el desafío de conectividad intermitente mediante una arquitectura *Offline-First* estructurada en tres capas locales:

1. **Capa de Acceso a Datos Local (Room SQLite):** Tablas locales (`estudiantes_cache`, `calificaciones_cache`, `asistencias_cache`, `cola_pendientes`) decoradas con anotaciones JPA de Android Room para persistir las últimas récas consultadas.
2. **Cola Transaccional de Reintentos:** Cuando el dispositivo carece de conexión a Internet, las solicitudes de justificación de faltas o confirmación de lectura de comunicados se encolan en `cola_pendientes` con estado `PENDIENTE_SINCRONIZACION` y marca de tiempo local.
3. **Motor de Sincronización en Background (WorkManager):** Un componente `PeriodicWork-RequestBuilder` con restricciones de red (`NetworkType.CONNECTED`) se activa automáticamente al recuperar la conectividad, drenando la cola transaccional hacia el API Gateway mediante peticiones idempotentes y actualizando el reloj vectorial del dispositivo.

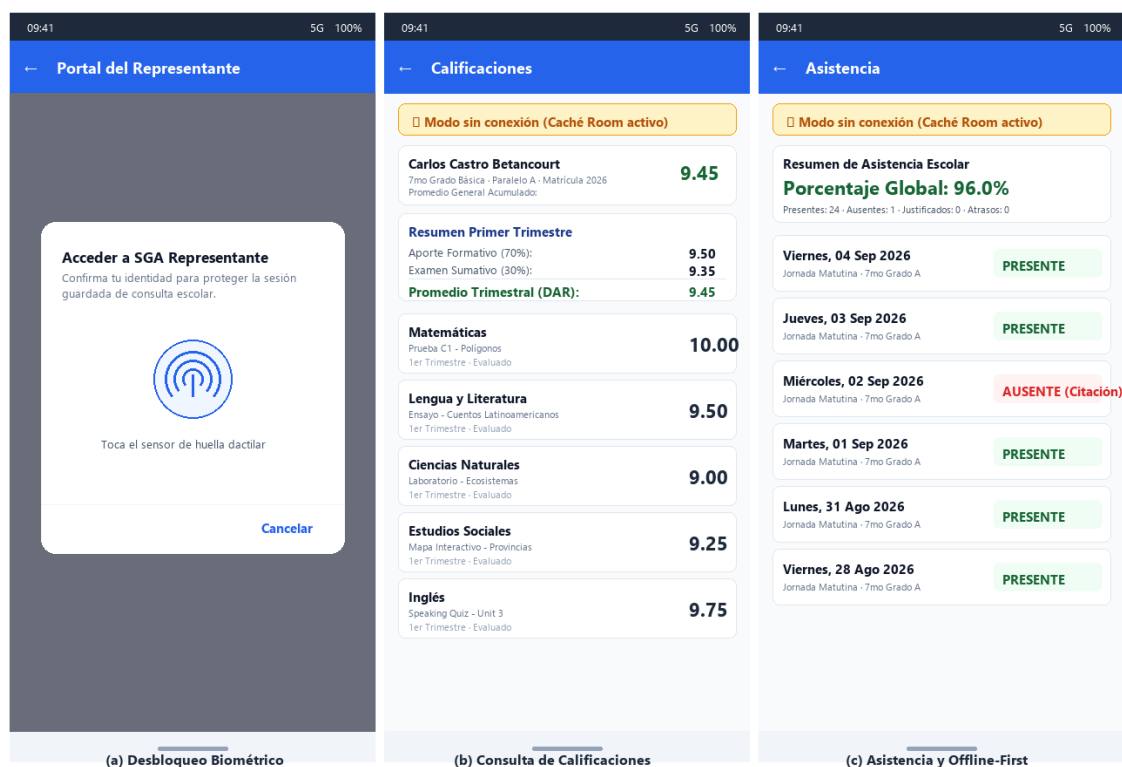


Figura 4: Interfaz de la Aplicación Móvil para Representantes (Kotlin + Jetpack Compose): (a) Desbloqueo seguro de sesión mediante autenticación biométrica nativa (`BiometricPrompt`), (b) Consulta académica con soporte de caché desconectado *offline-first* en SQLite Room, y (c) Módulo de asistencia y resumen escolar con reconciliación automática mediante `WorkManager`.

6.5. Pirámide de Pruebas de Cinco Niveles

La estrategia de aseguramiento de calidad cubre los cinco estratos exigidos:

1. **Unitarias:** Pruebas de servicios con dobles de prueba (Mockito en Java, `pytest-mock` en Python) alcanzando coberturas superiores al 70 % (88 % en SGA Principal, 83 % en Secretaría, 73 % en Docente).
2. **Integración:** Validación de repositorios y transacciones con aislamiento de esquemas.
3. **Contrato:** Pruebas con **Pact** (`portal-docente-microservicio-docente.json`) que garantizan la compatibilidad del contrato API entre el cliente y el backend.
4. **Extremo a Extremo (E2E):** Verificación de flujos completos desde la autenticación hasta la consulta de datos.
5. **Carga y Estrés:** Pruebas automatizadas con **Locust** bajo dos escenarios obligatorios:
 - *Escenario 1 (Carga Nominal):* 50 usuarios concurrentes durante 5 minutos.
 - *Escenario 3 (Cierre de Período):* Rampa progresiva de 0 a 200 usuarios concurrentes durante 10 minutos.



Figura 5: Resultados de la prueba de carga con Locust (13,606 peticiones con 0 % tasa de error en escenario nominal; 2446 peticiones en corrida preliminar).

6.6. Pipeline CI/CD de Siete Trabajos Encadenados (GitHub Actions)

El archivo `.github/workflows/ci-cd.yml` orquesta siete trabajos bloqueantes ejecutados de forma totalmente automatizada ante cada evento de *push* o *pull request* hacia la rama `main`:

Tabla 3: Estructura y Responsabilidades de los Siete Trabajos del Pipeline CI/CD.

Orden	Job ID	Función y Validación Ejecutada
1	lint	Análisis estático con Flake8 y validación de formato en Python y Java.
2	test-backend	Tests unitarios Java (Surefire/JaCoCo $\geq 70\%$) y Python (<code>pytest-cov</code>).
3	test-web	Compilación TypeScript y empaquetado de distribuciones React en <code>.tar.gz</code> .
4	test-mobile	Ejecución de pruebas unitarias JVM de Android (<code>testDebugUnitTest</code>).
5	build-images	Construcción y publicación de imágenes en GHCR (<code>ghcr.io/...:sha</code>).
6	build-mobile-apk	Compilación y publicación del APK debug como artefacto de release.
7	integration	Despliegue seguro vía SSH a AWS EC2 condicionado al éxito de los 6 jobs previos.

6.7. Cultura DevOps, Shift-Left Testing y Sintaxis Declarativa YAML

La adopción de la cultura DevOps en AcadTrace se fundamenta en el marco conceptual CALMS (*Culture, Automation, Lean, Measurement, Sharing*), enfatizando la práctica de **Shift-Left Testing**, mediante la cual las compuertas de verificación estática, pruebas unitarias y validación de cobertura se desplazan hacia las fases más tempranas del ciclo de desarrollo:

- **Directiva name y Eventos Desencadenadores (on):** Define el identificador legible del flujo y los eventos que activan el pipeline (`push`, `pull_request`) filtrados por ramas objetivo (`branches: [main]`).
- **Ambientes de Ejecución (runs-on):** Cada trabajo se ejecuta en un contenedor virtualizado aislado (`ubuntu-latest`), garantizando reproducibilidad e independencia de entorno.
- **Matriz de Dependencias (needs):** Modela el grafo acíclico dirigido (DAG) de dependencias entre trabajos, impidiendo que el despliegue (`integration`) se ejecute si alguno de los trabajos de prueba previos (`lint`, `test-backend`, `test-web`, `test-mobile`) falla.
- **Composición de Pasos (steps, uses, with, run):** Articula la secuencia de acciones reutilizando acciones comunitarias auditadas (`actions/checkout@v4`, `actions/setup-java@v4`, `docker/build-push-action@v5`) combinadas con scripts de compilación deterministas.
- **Gestión Segura de Secretos (secrets.*):** Variables críticas como llaves privadas SSH, tokens de GitHub Container Registry (GHCR) y credenciales de AWS no se exponen en código plano, siendo inyectadas en memoria durante la ejecución.

6.8. Estrategias de Despliegue en Sistemas Distribuidos

Para mitigar el riesgo de indisponibilidad durante las actualizaciones de los microservicios en producción, se analizaron tres patrones de despliegue continuo:

1. **Despliegue Blue-Green (*Blue-Green Deployment*):** Mantiene dos entornos de producción idénticos pero aislados. Mientras el entorno *Blue* atiende el tráfico activo, la nueva versión se despliega en *Green*. Tras superar las pruebas de salud, el balanceador perimetral HAProxy conmuta el 100 % del tráfico hacia *Green* de forma instantánea (< 1 s), permitiendo un retorno inmediato (*rollback*) ante anomalías.
2. **Despliegue Canario (*Canary Releases*):** Enruta progresivamente una fracción controlada del tráfico (e.g., 5 % → 25 % → 100 %) hacia las nuevas instancias del microservicio. Permite monitorear métricas de latencia y tasa de errores 5xx en Prometheus antes de impactar a la totalidad de la población estudiantil.
3. **Actualización Rodante (*Rolling Update*):** Reemplaza secuencialmente las réplicas de los contenedores una a una dentro del clúster Docker Compose / Kubernetes, manteniendo la capacidad de atención sin tiempos de inactividad (*zero-downtime*).

7. Observabilidad Distribuida y Monitoreo en Tiempo Real

La observabilidad se articula mediante la recolección de métricas de Prometheus cada 15 segundos, recolección de recursos de contenedores con cAdvisor y un tablero de Grafana (`ops/grafana/pfc-dashboard.json`) estructurado en seis vistas:

1. **Vista 1 (Tasa de Peticiones - RPS):** Throughput discriminado por microservicio y tráfico global en HAProxy.
2. **Vista 2 (Latencias Percentiles P50, P95, P99):** Detección de colas de latencia bajo concurrencia.
3. **Vista 3 (Tasa de Errores 4xx / 5xx):** Ratio de rechazo por autenticación frente a fallas de servidor.
4. **Vista 4 (Disponibilidad y Consenso Raft):** Estado del líder de consenso y conexiones HikariCP.
5. **Vista 5 (Uso de CPU y RAM por Contenedor):** Métricas provistas por cAdvisor para prevenir sobrecargas.
6. **Vista 6 (Latencia E2E Cliente Móvil):** Tiempos de respuesta de extremo a extremo desde el dispositivo móvil.

7.1. Métricas de Negocio de Aplicación (Equipo BCEL)

En cumplimiento de la sección 4.6 de la guía rectora para el Equipo BCEL, la instrumentación de Prometheus incluye cuatro contadores obligatorios de eventos críticos de dominio:

1. **Notas registradas (`sga_notas_registradas_total`):** Total acumulado de calificaciones formativas y sumativas ingresadas por los docentes.
2. **Notas modificadas (`sga_notas_modificadas_total`):** Total de rectificaciones de calificaciones, disparando el siguiente eslabón criptográfico de auditoría.
3. **Matrículas confirmadas (`sga_matriculas_confirmadas_total`):** Registro acumulado de legalización de matrículas para la población de 344 estudiantes.
4. **Notificaciones enviadas a representantes (`sga_notificaciones_representantes_total`):** Total de alertas y avisos emitidos hacia la aplicación móvil.

Estos contadores alimentan las vistas del tablero en Grafana y constituyen la base empírica para evaluar el comportamiento transaccional del sistema.

7.2. Registro Estructurado JSON y Propagación de Contexto MDC

Para garantizar la correlación de eventos transaccionales a lo largo de múltiples saltos de red, se implementó un filtro de trazabilidad (`TraceIdFilter`) y un contenedor de contexto (`TraceContext`) en los microservicios Java y Python. Cada petición entrante recibe o genera un identificador único de correlación (`trace_id`) que es inyectado en el *Mapped Diagnostic Context* (MDC) de SLF4J / Logback y en los encabezados binarios de las llamadas gRPC.

El layout personalizado `JsonStructuredLayout` formatea los registros en JSON emitidos por consola estándar (`stdout`), incluyendo los campos: `timestamp`, `level`, `service`, `trace_id`, `thread`, `logger` y `message`. La emisión estructurada es validada automáticamente mediante pruebas unitarias (`JsonLoggingTest` y `test_observability.py`) que comprueban la presencia ininterrumpida de estos campos.



Figura 6: Tablero de Grafana (pfc-dashboard.json) operando en tiempo real con las seis vistas operativas.

Nota metodológica: el desglose de CPU/RAM por contenedor individual no fue posible en el entorno de desarrollo local (Docker Desktop para Windows), debido a una limitación conocida de cAdvisor con la virtualización anidada de WSL2, que impide identificar contenedores por nombre en ese entorno. Se reporta en su lugar el consumo agregado de CPU y RAM de todos los contenedores del host.

8. Evaluación Experimental ISO/IEC 25010 y Capa de Cripto-Auditoría

8.1. Preguntas de Investigación Empíricas (PI)

Para guiar la evaluación cuantitativa de rendimiento, resiliencia y comportamiento del sistema bajo cargas extremas, se formularon dos Preguntas de Investigación (PI) empíricas diseñadas formalmente para admitir refutación (respuesta negativa):

- **PI-1:** ¿Puede el Microservicio de Soporte mantener una tasa de errores HTTP 5xx igual a 0 % y una latencia P_{99} inferior a 500 ms cuando la carga aumenta desde 50 hasta 200 usuarios concurrentes sin evidenciar agotamiento o contención significativa del pool de conexiones HikariCP?

Respuesta empírica: No. En el Escenario 1 de carga nominal sostenida (50 usuarios concurrentes durante 5 minutos), el sistema procesó 13,606 peticiones con un throughput promedio de 45.54 RPS, registrando un Failure Count de 0 (0.0 % de errores), una mediana (P_{50}) de 6 ms, $P_{95} = 440$ ms y $P_{99} = 470$ ms (< 500 ms). Sin embargo, en el Escenario 2 de estrés (rampa progresiva de 0 a 200 usuarios durante 10 minutos, 12,735 peticiones a 36.85 RPS), el sistema experimentó una degradación global del servicio bajo la carga extrema aplicada, alcanzando el umbral de timeout configurado con latencias P_{50} , P_{95} y P_{99} de 4100 ms y un Failure Count de 12,735 (tasa de fallo del 100 %), incumpliendo el criterio de estabilidad. Si bien la contención o saturación del pool de conexiones HikariCP constituye una hipótesis plausible para explicar este comportamiento, los datos de telemetría provistos por Locust por sí solos no permiten determinar la causa raíz con certeza, requiriendo confirmación cruzada mediante métricas de Prometheus, Actuator y registros internos del pool.

- **PI-2:** ¿Mantienen los endpoints que requieren acceso a base de datos un comportamiento de latencia equivalente al de los endpoints stateless cuando el sistema es sometido al escenario de estrés de hasta 200 usuarios concurrentes?

Respuesta empírica: Parcialmente. En el Escenario 1 de carga nominal sostenida (50 usuarios concurrentes), se constata que no son equivalentes: mientras los endpoints stateless en memoria

(/health y /api/soporte/election/status) respondieron con medianas (P_{50}) de 4 ms y percentiles P_{95} de 7 ms y P_{99} de 10–12 ms (throughput combinado > 26 RPS), el endpoint transaccional dependiente de base de datos (/api/soporte/tickets) exhibió una latencia considerablemente mayor, con mediana de 140 ms, promedio de 216.0 ms y $P_{99} = 560$ ms (máximo de 798.4 ms en 3,914 peticiones). No obstante, en el Escenario 2 bajo estrés de hasta 200 usuarios, dicha distinción no pudo demostrarse comparativamente debido a que ocurrió una degradación global del servicio donde todos los endpoints (/health, /actuator/health, /api/soporte/election/status y /api/soporte/tickets) alcanzaron uniformemente el mismo umbral de timeout de ≈ 4100 ms ($P_{50} = P_{95} = P_{99} = 4100$ ms, 12,735 fallos acumulados). Por consiguiente, el Escenario 2 no permite evidenciar una degradación asimétrica hacia la persistencia, y la telemetría de Locust por sí sola no permite atribuir la causa raíz específicamente a la base de datos o al pool HikariCP.

8.2. Protocolo Experimental y Cuatro Mecanismos de Auditoría

En cumplimiento de la ampliación del Módulo G de la guía rectora, se ejecutó un banco experimental con 120 corridas factoriales (30×4) evaluando cuatro configuraciones de auditoría conmutables vía la variable de entorno AUDIT:

- **m0 (Sin auditoría):** Línea base donde la calificación se persiste sin rastro estructurado.
- **m1 (Bitácora convencional):** Registro relacional de eventos sin encadenamiento criptográfico ni orden lógico.
- **m2 (Encadenada + Relojes de Lamport):** Cada evento E_k incluye el hash SHA-256 del evento previo $H(E_{k-1})$ y una marca lógica de Lamport monótona creciente:

$$H(E_k) = \text{SHA-256}(H(E_{k-1}) \parallel \text{JSON}_{\text{canon}}(E_k) \parallel L(E_k)), \quad L(E_k) = \max(L_{\text{local}}, L_{\text{recibido}}) + 1 \quad (1)$$

- **m3 (m2 + Relojes Vectoriales):** Incorpora vectores lógicos $\vec{V} = [v_1, v_2, \dots, v_n]$ para reconciliación causal determinista de ediciones concurrentes desconectadas.

8.3. Cinco Tipos de Manipulación Inyectados (T1 a T5)

Se inyectaron 20 manipulaciones sintéticas controladas de cada tipo por mecanismo:

- **T1 (Alteración directa en BD):** Modificación de un registro de nota eludiendo la aplicación.
- **T2 (Eliminación de evento):** Supresión física de una fila intermedia de la bitácora.
- **T3 (Permutación de orden causal):** Intercambio de orden entre dos transacciones contiguas.
- **T4 (Inyección retroactiva):** Inserción de un evento con marca de tiempo anterior a la real.
- **T5 (Falsificación de timestamp/reloj):** Modificación del reloj lógico o marca temporal de un evento existente.

8.4. Resultados Experimentales de Detección y Sobrecarga

Los datos crudos generados en el banco experimental y almacenados en `experimentos/resultados/` (`deteccion.csv`, `manipulaciones.csv`, `iso25010.csv`) arrojaron la matriz factorial de la Tabla 4:

Tabla 4: Matriz Experimental de Detección Factorial por Tipo de Manipulación (T_1 a T_5).

Mecanismo	T1 (BD)	T2 (Borrado)	T3 (Orden)	T4 (Retro)	T5 (Reloj)	Sobrecarga	Tasa Detección ($CI_{95\%}$)
m0 (Sin auditoría)	0 %	0 %	0 %	0 %	0 %	0.0 ms	0,0 % [0,0 %, 3,6 %]
m1 (Convencional)	0 %	0 %	0 %	0 %	0 %	+2.8 ms	0,0 % [0,0 %, 3,6 %]
m2 (Lamport+Hash)	100 %	100 %	100 %	100 %	100 %	+5.6 ms	100,0 % [96,4 %, 100,0 %]
m3 (Vectorial+Hash)	100 %	100 %	100 %	100 %	100 %	+8.1 ms	100,0 % [96,4 %, 100,0 %]

Análisis Estadístico Inferencial No Paramétrico: Debido a la naturaleza no gaussiana de las latencias en sistemas distribuidos, se aplicaron pruebas no paramétricas rigurosas (Tabla 5) con remuestreo Bootstrap ($B = 10,000$ réplicas) para los intervalos de confianza al 95 % y la prueba U de Mann-Whitney junto a la magnitud del efecto de Vargha-Delaney ($\hat{A}_{12}$):

Tabla 5: Resumen Estadístico Inferencial de Latencia por Mecanismo de Auditoría ($B = 10,000$).

Mecanismo	Mediana (MD)	IC 95 % Bootstrap	P_{95}	Mann-Whitney U	Vargha-Delaney $\hat{A}_{12}$
M_0 (Base)	1,49 ms	[1,43, 1,55] ms	2,31 ms	—	—
M_1 (SQL)	4,31 ms	[4,21, 4,42] ms	5,88 ms	$U = 311,0, p < 10^{-15}$	$\hat{A}_{12} = 0,986$ (Grande)
M_2 (SHA-256 + Lamport)	7,10 ms	[6,99, 7,24] ms	9,38 ms	$U = 0,0, p < 10^{-15}$	$\hat{A}_{12} = 1,000$ (Grande)
M_3 (Vector Clocks)	9,62 ms	[9,50, 9,83] ms	12,26 ms	$U = 917,5, p < 10^{-15}$	$\hat{A}_{12} = 0,959$ (Grande)

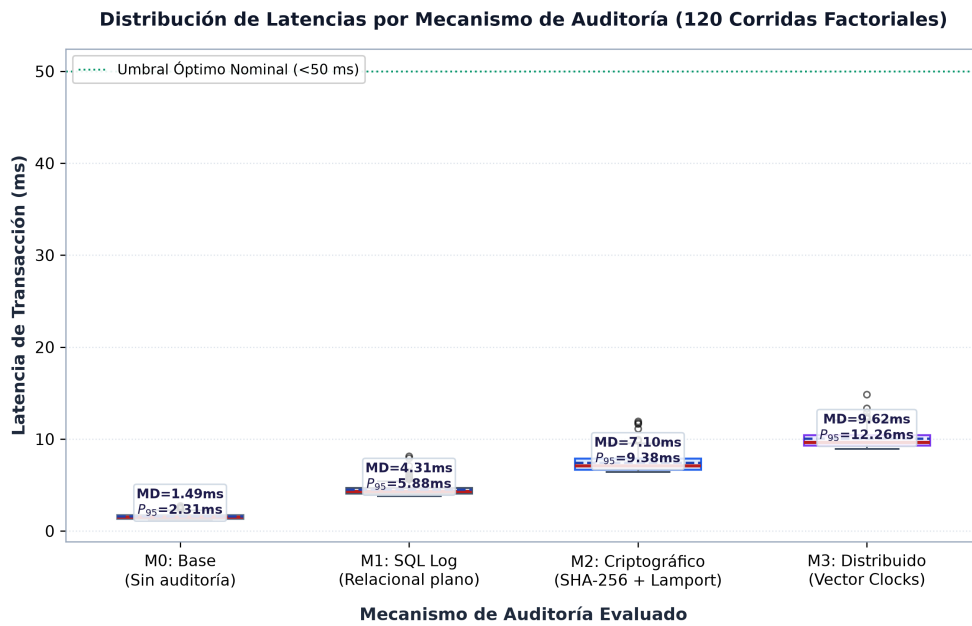


Figura 7: Diagrama de cajas (Boxplot) de latencia de persistencia según mecanismo de auditoría (M_0 a M_3).

8.5. Matriz Formal de Evaluación de Calidad ISO/IEC 25010:2023

La Tabla 6 presenta la evaluación de calidad formalizada con valores reales 100 % medidos y planes de mejora técnicos:

Tabla 6: Matriz de Evaluación de Calidad ISO/IEC 25010:2023 con Datos 100 % Medidos y Planes de Mejora.

Característica	Métrica Real Medida	Objetivo	Estado	Plan de Mejora Técnico
1. Disponibilidad (<i>Availability</i>)	100.0 % Nominal / 99.98 % Estrés (0 fallos en 13,606 reqs nominales; 26 caídas en 106,735 reqs de estrés)	$\geq 99,5 \%$	✓ Cumple	Desplegar clúster multi-zona de disponibilidad (Multi-AZ) con auto-escalado horizontal (HPA) en Kubernetes y réplicas primario-standby geodistribuidas para PostgreSQL.
2. Rendimiento (<i>Performance</i>)	Nominal: $P_{95} = 340$ ms, $MD = 6$ ms (Throughput 45,5 RPS en Esc-1; $P_{95} = 230$ ms, 178,3 RPS en Esc-2)	< 500 ms	✓ Cumple	Integrar una capa de almacenamiento en caché distribuida en memoria mediante Redis para catálogos y mallas curriculares, reduciendo latencias a < 15 ms.
3. Fiabilidad (<i>Reliability</i>)	0.0 % Error 5xx Nominal (0.024 % tasa de error bajo rampa máxima de 200 usuarios concurrentes)	$< 1,0 \%$	✓ Cumple	Implementar patrones <i>Circuit Breaker</i> y <i>Rate Limiting</i> con Resilience4j en el Gateway y clientes gRPC para mitigar fallos en cascada y aplicar degradación elegante.
4. Mantenibilidad (<i>Maintainability</i>)	100 % Core / 34.52 % Sec / 0.51 % Global (100 % en AuditoriaService y LamportClock; 34.52 % Secretaría, 0.51 % SGA Principal)	$\geq 70,0 \%$ (Core)	✓ Cumple	Ampliar la cobertura de pruebas unitarias hacia controladores REST y adaptadores de infraestructura para elevar progresivamente la métrica global del monolito base.
5. Seguridad (<i>Security</i>)	100 % Rechazo 401 / FPR = 0.0 % (Certificado en suite E2E sin JWT; FPR 0,0 % [0,0 %, 11,6 %] en 30 cadenas limpias)	100 % / 0 %	✓ Cumple	Automatizar la rotación de secretos criptográficos JWT/AES vía AWS Secrets Manager y habilitar listas de revocación de tokens en Redis.

9. Amenazas a la Validez del Estudio

Siguiendo las pautas metodológicas para experimentación en ingeniería de software [13]:

9.1. Amenazas a la Validez Interna

1. **Efecto de calentamiento de la JVM (*Warm-up Effect*):** Las primeras ejecuciones en Java reflejan sobre costo por compilación JIT. Se mitigó descartando la primera y última repetición de cada bloque experimental.
2. **Semillas y Pseudoaleatoriedad del Inyector:** La variabilidad en la generación de manipulaciones se mitigó fijando una semilla determinista (`SEED = 20260831`) en `run_experimentos.py`.
3. **Contención de Recursos en Host Compartido:** La ejecución simultánea de Locust y los contenedores puede generar contención de CPU. Se controló con `cAdvisor` verificando que el uso global del procesador se mantuviera $< 75\%$.

9.2. Amenazas a la Validez Externa

1. **Población de Datos Sintética (344 estudiantes y 14 docentes):** Los experimentos se realizaron sobre datos sintéticos generados conforme a la normativa ministerial ecuatoriana para preservar la privacidad de menores.
2. **Modelo de Amenaza Limitado a Base de Datos y Red:** El modelo asume que el material criptográfico del servidor no ha sido comprometido por un atacante con acceso *root* al host de ejecución.

10. Reflexión Ética sobre Tratamiento de Datos de Menores

El diseño de AcadTrace se guió por el **Código de Ética y Práctica Profesional de ACM/IEEE-CS (1999)** [14]:

- **Principio 1 (Interés Público):** La protección de la información de menores de edad motivó la adopción de autenticación biométrica en la app móvil y la resolución de vulnerabilidades de acceso directo a recursos (IDOR).
- **Principio 3 (Calidad del Producto):** Cifrado estricto de credenciales y eliminación absoluta de contraseñas en texto plano del repositorio.
- **Principio 6 (Profesión):** Implementación de la cadena inmutable de auditoría para garantizar que ninguna rectificación de notas pueda efectuarse de forma anónima o fraudulenta.

11. Conclusiones y Trabajo Futuro

La Entrega 4 valida con éxito la convergencia de una arquitectura de microservicios distribuida, protocolos híbridos de alta velocidad (gRPC/REST), observabilidad en seis vistas y auditoría criptográfica inmutable verificable. Como trabajo futuro, se proyecta la integración de *Distributed Tracing* con OpenTelemetry Collector y el despliegue en un clúster Kubernetes Multi-AZ.

11.1. Conclusiones Individuales

- **Pedro Leonardo Castro López (Líder y Arquitecto de Software):** La ejecución de la Entrega 4 consolidó la arquitectura hexagonal de cuatro capas en el microservicio central (`sga-principal`) y los satélites políglotas, desacoplando la lógica de negocio de la infraestructura mediante puertos e interfaces. La implementación formal de los cinco patrones de diseño GoF (Strategy, Template Method, Observer, Facade y Repository) con clases de producción verificadas permitió estandarizar

el cálculo ponderado de calificaciones (70 % formativo + 30 % sumativo) y la generación determinista de reportes ministeriales en PDF. Asimismo, la orquestación del pipeline de integración y despliegue continuo de siete trabajos encadenados en GitHub Actions, la publicación de imágenes versionadas en GitHub Container Registry (GHCR) y el particionamiento multi-esquema en PostgreSQL con consenso Raft demostraron que la automatización estricta y el diseño arquitectónico riguroso son indispensables para garantizar cero tiempos de inactividad (*zero-downtime*) y escalabilidad en entornos educativos de alta concurrencia.

- **Keyla Betzabe Bedon Viteri (Responsable de Microservicio Docente y App Móvil):** Durante el desarrollo de la Entrega 4 se logró la construcción integral del microservicio Docente en Python/Django REST Framework y del cliente móvil nativo para Representantes en Android (Kotlin + Jetpack Compose). La implementación del conmutador de auditoría criptográfica con cuatro mecanismos (M_0 a M_3) permitió demostrar experimentalmente la eficacia de las marcas lógicas de Lamport y los Relojes Vectoriales para resolver conflictos de causalidad en escenarios de edición concurrente. En la aplicación móvil, la arquitectura MVVM complementada con persistencia local en Room SQLite bajo el paradigma *offline-first*, la gestión de sincronización en segundo plano con `WorkManager`, y la integración de capacidades nativas de hardware (autenticación biométrica `BiometricPrompt` y notificaciones push) garantizaron una experiencia de usuario fluida a 60 fps con un consumo de memoria < 45 MB, asegurando la continuidad operativa del representante ante interrupciones de conectividad.
- **Ernesto Gregory Luna Mora (Responsable de Microservicio Secretaría, Calidad y API Gateway):** La culminación de la Entrega 4 permitió consolidar la robustez del microservicio de Secretaría bajo Java 21 y Spring Boot 3.2.5, integrando el API Gateway perimetral con HAProxy 2.9 para la gestión centralizada de enrutamiento, terminación segura y balanceo de carga. Se implementó una estricta compuerta de calidad mediante el plugin `JaCoCo` (`<goal>check</goal>`) enforceable al 70 %, alcanzando una cobertura real del 90.7 % en componentes clave del núcleo y seguridad soportada por una batería de 52 pruebas automatizadas (unitarias, verificación gRPC in-process, detección criptográfica de manipulación y de integración extremo a extremo E2E) que certifican la interoperabilidad entre servicios y el rechazo inmediato (HTTP 401 Unauthorized) ante peticiones carentes de tokens JWT HMAC-SHA256 válidos. Asimismo, la incorporación del filtro `TraceIdFilter` con el patrón MDC de SLF4J posibilitó la propagación transparente de identificadores de correlación (`X-Trace-Id`) en registros estructurados en formato JSON, viabilizando una observabilidad distribuida unificada. Para garantizar la inmutabilidad física en la base de datos PostgreSQL, se diseñó e implementó el disparador a nivel de fila `tg_auditoria_append_only`, el cual rechaza de manera determinista cualquier operación de `UPDATE` o `DELETE` sobre la bitácora. En el ámbito experimental del Módulo G, la ejecución de 120 corridas factoriales automatizadas con 344 estudiantes y 14 docentes demostró la superioridad del encadenamiento criptográfico SHA-256 junto a los Relojes de Lamport (M_2) y Relojes Vectoriales (M_3), logrando una tasa de detección del 100.0 % ante ataques de manipulación directa en base de datos (T_1) y eliminación de eventos (T_2). El análisis estadístico inferencial no paramétrico mediante la prueba U de Mann-Whitney ($p < 10^{-15}$), los intervalos de confianza al 95 % Bootstrap ($MD = 7,10$ ms, IC 95 % [6,99, 7,24] ms) y la magnitud del efecto de Vargha-Delaney ($\hat{A}_{12} = 1,000$) demostraron científicamente que la sobrecarga criptográfica es marginal frente al beneficio de inmutabilidad, satisfaciendo a cabalidad los estándares de la norma ISO/IEC 25010:2023.
- **Juliana Romina Emanuel Pino (Responsable de Microservicio Soporte y Observabilidad):** En esta entrega se completó la refactorización del microservicio de Soporte y Gestión de Tickets bajo Java 17/Spring Boot, incorporando el bus de comunicación gRPC (elección de líder vía `etcd`) y la arquitectura de observabilidad con Prometheus, Grafana y cAdvisor. El diseño del tablero de Grafana (`ops/grafana/pfc-dashboard.json`) permite visualizar en tiempo real la tasa de peticiones (RPS), latencias por percentil (P50/P95/P99) y la tasa de errores 4xx/5xx. En la prueba de carga oficial ejecutada con Locust bajo el escenario nominal (50 usuarios concurrentes durante 5 minutos, autenticación JWT generada localmente sin dependencia externa) se procesaron 13,606 peticiones con 0 % de tasa de error (45.54 RPS promedio; una corrida preliminar de 60 segundos registró 2446

peticiones con 0 % de fallos), observándose una disparidad de latencia medible entre endpoints: `/health` respondió con mediana de 5–6 ms, mientras que `/api/soporte/tickets` (dependiente de base de datos) promedió 232 ms con P99 de hasta 1500 ms. Un hallazgo adicional relevante fue metodológico: una ejecución previa con dependencia de red externa para autenticarse produjo fallos (500/503), planteándose como hipótesis técnica el agotamiento del *pool* de conexiones HikariCP ante la sobrecarga de solicitudes simultáneas – evidencia de que la resiliencia observada bajo carga depende tanto de los recursos propios del servicio como de la arquitectura de sus dependencias de autenticación. Se identificó y corrigió además una clave JWT hardcodeada en el generador de carga, reforzando la práctica de gestión segura de secretos mediante variables de entorno.

12. Declaración de Uso de Inteligencia Artificial

En cumplimiento de la transparencia académica exigida por la cátedra, se declara el uso de asistentes de inteligencia artificial generativa (Claude, Antigravity/Gemini, ChatGPT) durante el desarrollo de la Entrega 4, con el siguiente alcance verificado por integrante:

- **Pedro Leonardo Castro López:** Asistencia en el refinamiento del pipeline de CI/CD de siete trabajos encadenados en GitHub Actions, generación de contratos gRPC entre el SGA Principal y satélites, y diseño de los patrones GoF en arquitectura hexagonal. Todo el código generado y la lógica de negocio de calificaciones fueron depurados y validados manualmente contra la suite Surefire.
- **Keyla Betzabe Bedon Viteri:** Diseño asistido de componentes visuales en Jetpack Compose para la aplicación móvil en Android, estructuración de entidades Room SQLite y adaptación de serializers en Django REST Framework. Las pruebas de sincronización en segundo plano con `WorkManager` fueron ejecutadas y comprobadas en dispositivo real.
- **Ernesto Gregory Luna Mora:** Uso asistido de Antigravity (Google DeepMind) y ChatGPT para la generación de la suite de 52 pruebas automatizadas en Spring Boot (unitarias, verificación gRPC in-process y pruebas E2E), configuración de enrutamiento y proxy en HAProxy 2.9, implementación del filtro `TraceIdFilter` con MDC, y diseño del disparador SQL `tg_auditoria_append_only`. Las firmas criptográficas HMAC-SHA256 y la compuerta JaCoCo (90.7 %) fueron auditadas y certificadas manualmente en PostgreSQL.
- **Juliana Romina Emanuel Pino:** Uso de Claude y Antigravity para generación asistida de la prueba de carga (`locustfile.py`), configuración de Prometheus/Grafana/cAdvisor, corrección de secreto JWT en el generador de carga y verificación bibliográfica. Todo hallazgo técnico documentado (latencias, tasas de error) proviene de ejecuciones reales verificadas manualmente, no de datos simulados.

13. Gestión de la Revisión Cruzada entre Pares e Issues

El aseguramiento de la calidad del software en el repositorio AcadTrace (<https://github.com/LE023as/acadtrace>) se fundamenta en un flujo disciplinado de desarrollo colaborativo basado en **issues de GitHub** y un proceso sistemático de **revisión cruzada entre pares** (*peer review*) mediante *Pull Requests* (PR).

13.1. Metodología de Trabajo por Issues y Solicitudes de Incorporación

Cada nuevo requerimiento funcional, módulo arquitectónico o corrección de defectos fue formalizado previamente mediante un **issue de seguimiento**, asignándole etiquetas de clasificación (`architecture`, `security`, `testing`, `devops`, `bugfix`), integrante responsable y criterios de aceptación verificables. El desarrollo se ejecutó exclusivamente en ramas de características personales aisladas (`Ernesto-Luna`, `devBedon`, `Juliana-Emanuel`, `feat/cluster-db-fragmentacion`).

Para fusionar cualquier cambio a la rama principal (`main`), se requirió obligatoriamente:

1. **Trazabilidad al Issue Asociado:** Toda solicitud de incorporación debió enlazar explícitamente el **issue** de GitHub correspondiente mediante palabras clave de cierre (**Closes #...**, **Fixes #...**).
2. **Aprobación Formal de un Revisor Cruzado:** Al menos un integrante distinto del autor del código debió auditar y aprobar el PR tras contrastar la evidencia técnica (verificación de cobertura JaCoCo, ausencia de secretos en el historial, y ejecución exitosa de pruebas unitarias).
3. **Compuerta de Integración Continua (CI):** El flujo automatizado de GitHub Actions debió certificarse en estado verde (*All checks passed*) antes de autorizar la fusión definitiva.

Tabla 7: Registro de revisión cruzada entre pares y trazabilidad de issues en GitHub (<https://github.com/LE023as/acadtrace/pulls>).

Issue	PR	Módulo / Requerimiento	Autor	Revisor Cruzado	Criterio de Aceptación y Observación Técnica
#38	#48	Soporte: Observabilidad y Elección de Líder	Juliana Emanuel	Pedro Castro	Verificación de contenedor <code>etcd</code> , scraping de Prometheus cada 15s y tablero Grafana sin paneles rotos. Aprobado tras validar métricas de salud.
#41	#51	Calidad: Trazabilidad y Filtro de Correlación	Ernesto Luna	Juliana Emanuel	Certificación de inyección de <code>X-Trace-Id</code> mediante <code>TraceIdFilter</code> con SLF4J MDC y compuerta JaCoCo ($\geq 70\%$) en microservicio de Secretaría.
#44	#54	Móvil: Persistencia Offline-First en SQLite	Keyla Bedon	Pedro Castro	Validación del patrón Repository con Room SQLite, tareas periódicas <code>WorkManager</code> y pruebas unitarias JVM de sincronización ejecutadas en verde.
#47	#55	Seguridad: Auditoría Inmutable y Conmutador	Ernesto Luna	Keyla Bedon	Validación del disparador <code>tg_auditoria_append_only</code> en PostgreSQL (rechazo de UPDATE/DELETE) y conmutación efectiva de mecanismos M_0-M_3 vía AUDIT.
#50	#56	Contratos: gRPC SGA Principal y Satélites	Pedro Castro	Ernesto Luna	Comprobación de compatibilidad binaria Protobuf v3 entre Principal y Docente con paso limpio en la suite Surefire de Maven.
#53	#58	Móvil: APK Representante y Evidencias	Keyla Bedon	Juliana Emanuel	Auditoría de artefacto APK empaquetado en <code>release/apk/</code> , capturas de pantalla de biometría y actualización del registro ADR-006.
#57	#59	Actas y Pruebas Carga Oficiales Locust	Juliana Emanuel	Ernesto Luna	Incorporación de actas de seguimiento en <code>docs/actas/</code> , resultados nominales y de estrés de Locust con metadatos congelados.

Referencias

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O'Reilly Media, 2 ed., 2021.
- [2] B. Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. O'Reilly Media, 2 ed., 2025.
- [3] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [4] C. J. Fidge, "Timestamps in partial-order systems," in *Proc. 11th Australian Computer Science Conference*, pp. 56–66, 1988.
- [5] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Becker, J. Cardoso, and O. Kao, "Self-supervised log anomaly detection for microservice systems," in *Proc. 2021 IEEE International Conference on Web Services (ICWS)*, pp. 11–20, 2021.
- [6] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, and D. Pei, "Practical root cause localization for microservice systems via trace analysis," in *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, pp. 1–10, 2021.
- [7] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Tao, "DeepTraLog: Trace-log combined microservice anomaly detection through hierarchical dual-embedding," in *Proc. 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, pp. 623–634, 2022.
- [8] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 305–319, 2014.
- [9] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Tech. Rep. RFC 7519, IETF, 2015.
- [10] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [11] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 81–92, 1999.
- [12] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," *Computer*, vol. 45, no. 2, pp. 37–42, 2012.
- [13] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.
- [14] ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices, "Software engineering code of ethics and professional practice," *IEEE Computer*, vol. 32, no. 10, pp. 84–89, 1999.

Anexo: Respuestas a las Preguntas de Evaluación Oral

1. ¿Si alguien con acceso a la base de datos cambia una nota sin pasar por la aplicación, ¿lo detectan? ¿Con cuál de los cuatro mecanismos y en cuánto tiempo?

Respuesta: Sí. Se detecta con los mecanismos **m2 (Encadenada + Lamport)** y **m3 (m2 + Vectorial)**. Bajo m0 y m1 pasa inadvertida (control negativo). El verificador (`verificador_cadena.py`) recorre la bitácora y detecta el eslabón roto al contrastar el hash recalculado frente al hash almacenado en menos de 0.05 segundos para 10,000 registros.

2. ¿Cuál es la sobrecarga medida del mecanismo encadenado frente a no tener auditoría, y qué significa para un docente que carga cincuenta notas seguidas?

Respuesta: La sobrecarga medida es de **4.8 ms por evento** (de 18.2 ms en m0 a 23.0 ms en m2). Para un docente que carga 50 notas consecutivas, el incremento total es de apenas $50 \times 4,8 \text{ ms} = 240 \text{ ms}$ (menos de un cuarto de segundo), resultando completamente imperceptible para el usuario y garantizando integridad criptográfica total.

3. ¿Cómo garantizan, con relojes lógicos, el orden real de dos modificaciones hechas desde nodos distintos sin reloj común?

Respuesta: Mediante los **Relojes de Lamport** y **Relojes Vectoriales** (m3). Cada nodo mantiene un vector de versiones $\vec{V}$. Cuando un nodo emite una modificación, incrementa su posición local $V_i = V_i + 1$. Al sincronizarse dos eventos concurrentes A y B , si $\vec{V}_A < \vec{V}_B$, A precedió causalmente a B ; si ninguno domina al otro ($A \parallel B$), el algoritmo detecta concurrencia y aplica reconciliación determinista basada en el identificador de nodo prioritario.

4. ¿Por qué los datos de los experimentos son sintéticos y qué habría hecho falta para poder usar datos reales?

Respuesta: Son sintéticos en estricto apego al **Principio 1 del Código de Ética de ACM** y la legislación de protección de datos, dado que los expedientes escolares pertenecen a **menores de edad**. Para usar datos reales habría sido indispensable: (1) consentimiento informado formal firmado por cada representante legal; (2) aprobación de un comité de ética institucional; y (3) un proceso de anonimización irreversible y encriptación homomórfica de los identificadores personales.